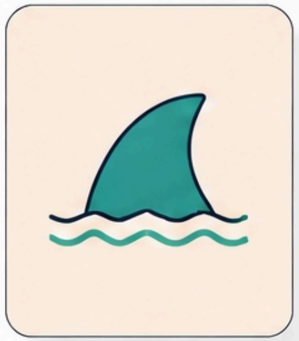
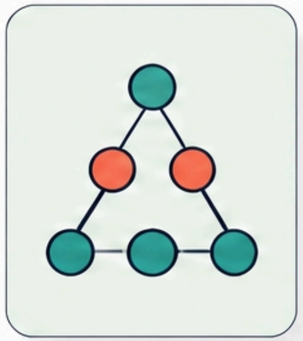
NETWORKING TOOLS HANDBOOK



Illustrated Tutorials · Running Examples · Complete Command Reference



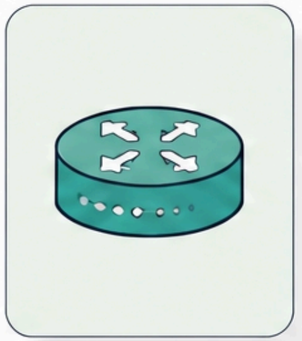
Wireshark



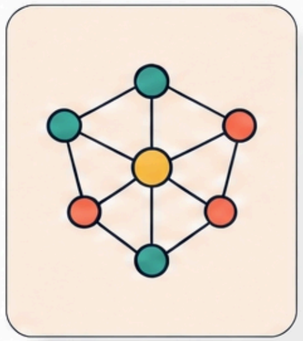
Mininet



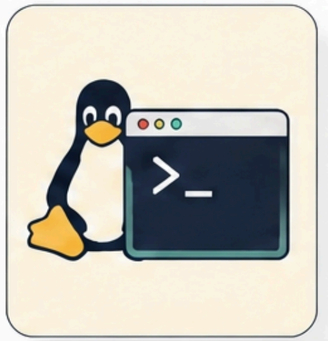
iperf3



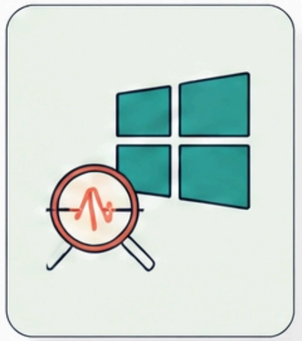
Cisco Packet Tracer



GNS3



Linux Utilities



Npcap

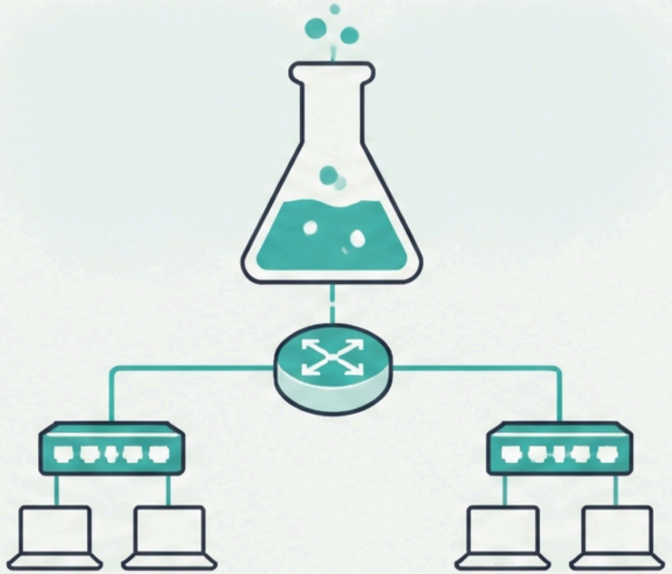
Computer Networks Lab by Nabajyoti Medhi

Why Master These Tools?



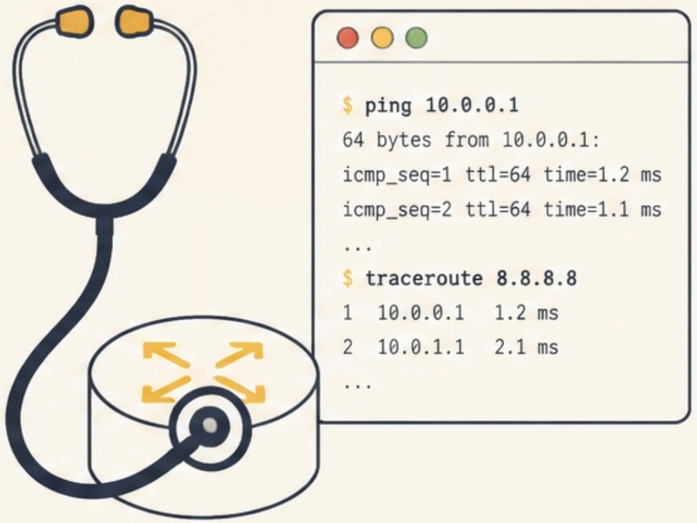
SEE THE INVISIBLE –

Wireshark reveals every packet on the wire



BUILD BEFORE YOU BREAK –

Mininet & GNS3 emulate whole networks safely



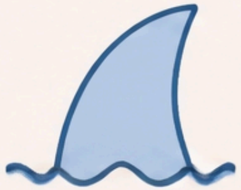
DIAGNOSE LIKE A PRO –

Linux utilities are the industry standard

“ Networks fail. Great engineers see why. ”

The Networking Tools Landscape

1

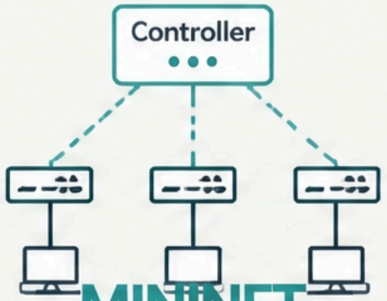


```
00 1A 2B 3C
45 00 00 54
08 00 45 00
```

WIRESHARK

Deep packet capture & protocol analysis

2



MININET

SDN emulation on one Linux box

3



TCP	1.24 Gbps
UDP	950 Mbps
Loss	0.12%
Jitter	1.8 ms

IPERF3

TCP/UDP throughput, loss, jitter measurement

4

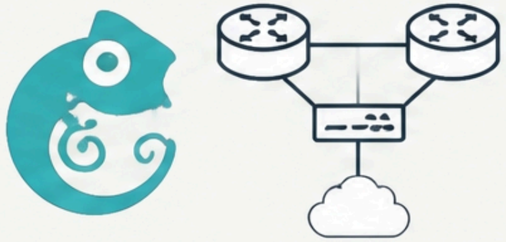


CISCO

PACKET TRACER

Beginner-friendly Cisco topology simulator

5



GNS3

Multi-vendor router emulation for CCNA/CCNP

6

```
$ ip addr
$ ss -tuln
$ ping 8.8.8.8
$ tcpdump -i eth0
$ nmap -sS 10.0.0.1
$ dig example.com
```

LINUX UTILS

ip · ss · ping · tcpdump · nmap · dig

7



NPCAP

Windows packet capture driver

8

```
$ tshark -i eth0 -w cap.pcap
$ tshark -r cap.pcap -Y "tcp"
$ tshark -T fields -e ip.src -e
ip.dst -r cap.pcap
```

TSHARK

Wireshark from the command line

= capture/analysis = simulation = measurement

The Mental Model: Build → Measure → Capture

the engineer's workflow

1



BUILD TOPOLOGY

Packet Tracer · GNS3 · Mininet

2



MEASURE PERFORMANCE

iperf3 · ping · mtr

3



CAPTURE & TROUBLESHOOT

Wireshark · tcpdump · tshark

Start with topology building, move to measurement, then capture and troubleshood traffic.

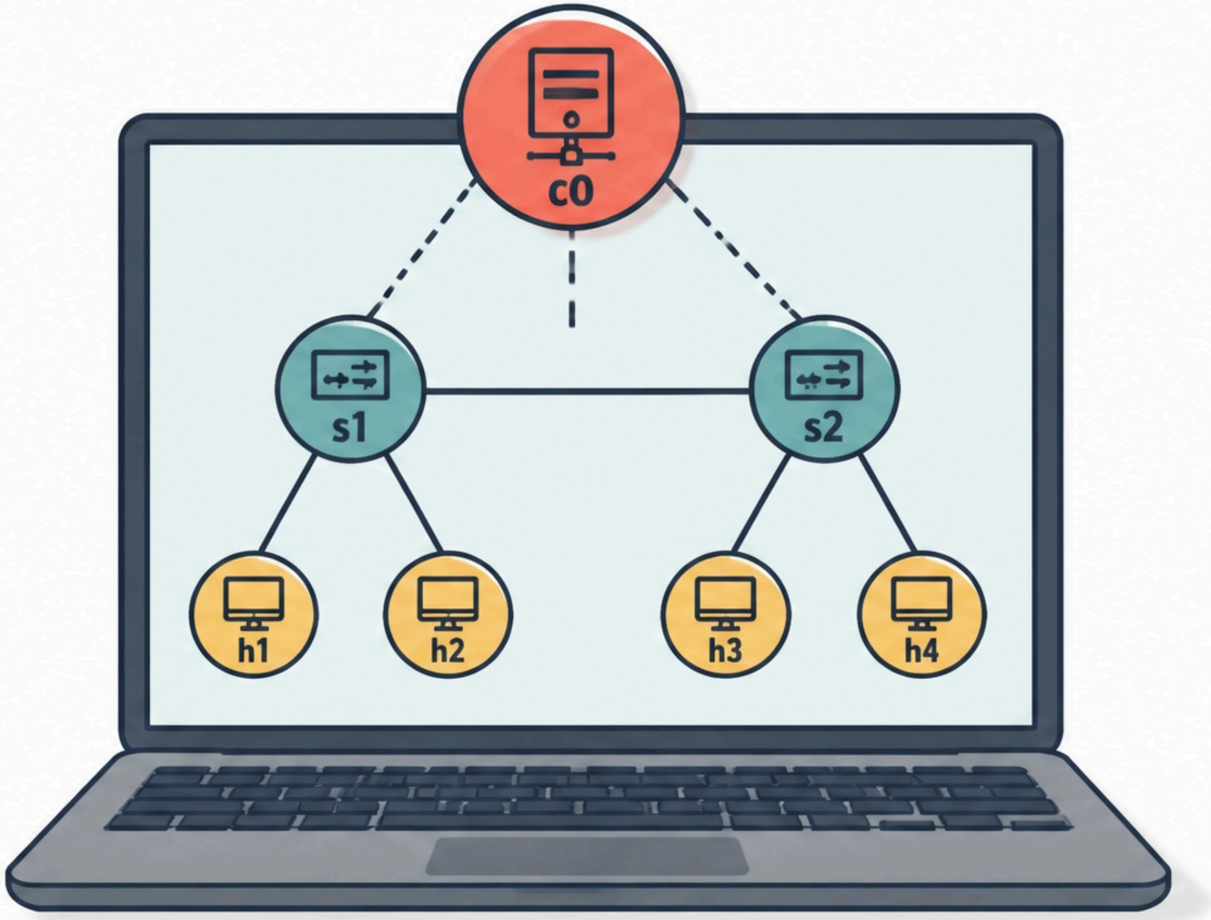
Recommended 8-Week Learning Roadmap

Start with Packet Tracer, end with Security

1. **Packet Tracer —** Build your first LAN
2. **Linux utilities —** ip, ping, ss, traceroute
3. **Wireshark basics —** capture your first packets
4. **Wireshark filters —** display & capture
5. **iperf3 —** measure throughput & loss
6. **Mininet —** emulate SDN topologies
7. **GNS3 —** multi-router routing labs (OSPF, BGP)
8. **Security & Packet Forensics —** nmap, tcpdump

SECTION 2 · MININET

Advanced Networking / SDN Experiments



Used at Stanford / Berkeley

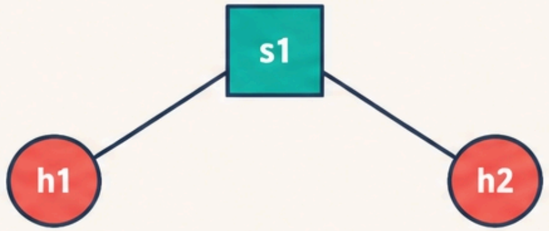
Backbone of SDN research

Runs on any Linux VM

> *"Emulate a whole network — controllers, switches, hosts, links — on one Linux machine."*

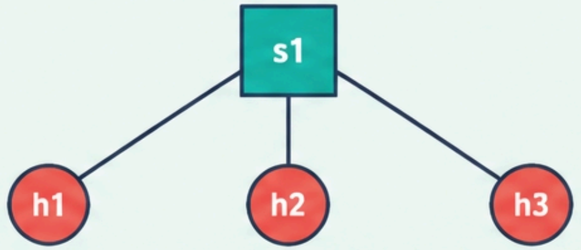
Mininet Topologies — Choose Your Shape

1 MINIMAL topology



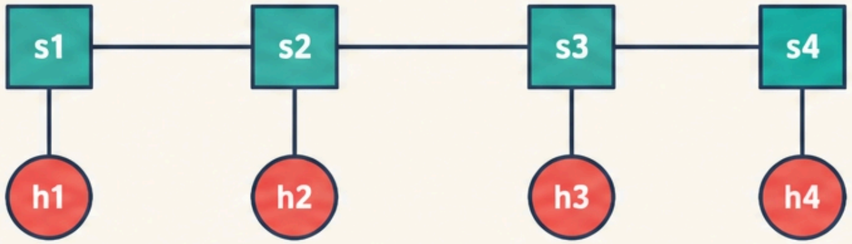

```
sudo mn --topo minimal
```

2 SINGLE topology



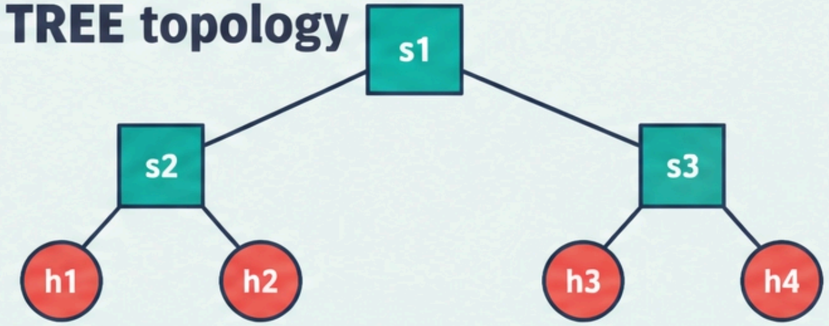

```
sudo mn --topo single,3
```

3 LINEAR topology




```
sudo mn --topo linear,4
```

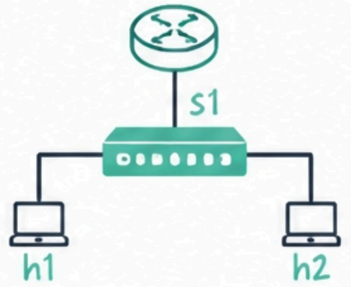
4 TREE topology




```
sudo mn --topo tree,depth=2,fanout=2
```

Add realism: `--link tc,bw=10,delay=10ms,loss=1` to constrain bandwidth, delay, and loss

Mininet CLI • The mininet> Prompt



INSPECT THE NETWORK

```
mininet> help
mininet> nodes
mininet> net          # show links
mininet> dump         # detailed info
mininet> ports        # switch ports
mininet> links
mininet> intfs
```

RUN TESTS & COMMANDS

```
mininet> pingall
mininet> pingallfull
mininet> iperf h1 h2
mininet> iperfudp 10M h1 h2
mininet> link s1 h1 down   <-- chaos engineering!
mininet> link s1 h1 up
mininet> xterm h1 h2
mininet> h1 ping -c 3  h2
mininet> h1 ifconfig h1-eth0
mininet> py net.hosts
mininet> exit
```

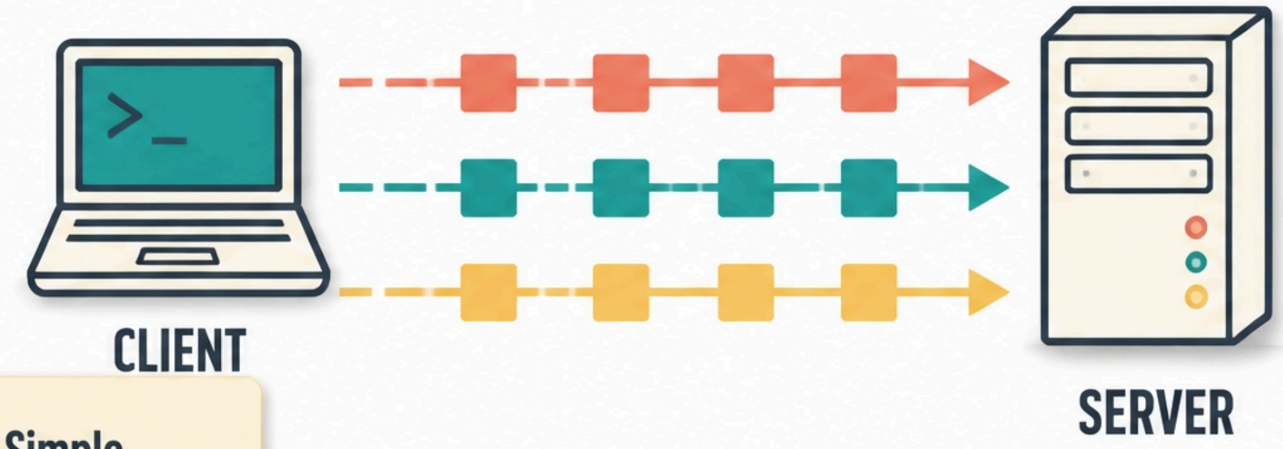
Batch tests from bash: `sudo mn --topo linear,4 --test pingall`

SECTION 3 · IPERF3

Network Performance Measurement · TCP · UDP · Jitter · Loss

> *How fast is your link, really? And what happens under load?*





Measures throughput, loss, jitter

TCP + UDP + IPv6

Simple client/server model

iperf3 · TCP Measurement · Server & Client

SERVER SIDE

```
$ iperf3 -s                  # default port 5201
$ iperf3 -s -p 5202          # custom port
$ iperf3 -s -D               # daemon mode
$ iperf3 -s -1               # exit after one client
$ iperf3 -s -B 10.0.0.5      # bind to IP
$ iperf3 -s -i 1             # report every 1s
```

TCP stream

CLIENT SIDE — TCP

```
$ iperf3 -c 192.168.1.20
$ iperf3 -c 192.168.1.20 -t 30     # 30-second test
$ iperf3 -c 192.168.1.20 -i 2      # print every 2s
$ iperf3 -c 192.168.1.20 -P 4      # 4 parallel streams
$ iperf3 -c 192.168.1.20 -R        # reverse (server → client)
$ iperf3 -c 192.168.1.20 --bidir   # both directions
$ iperf3 -c 192.168.1.20 -w 1M     # TCP window size
$ iperf3 -c 192.168.1.20 -M 1400   # MSS
$ iperf3 -c 192.168.1.20 -O 5      # omit first 5s
```

iperf3 · UDP, Advanced Scenarios & Automation



UDP (loss & jitter)

```
$ iperf3 -c 192.168.1.20 -u -b 20M
  # 20 Mbit/s UDP

$ iperf3 -c 192.168.1.20 -u -b 0
  # UDP no limit

$ iperf3 -c 192.168.1.20 -u -b 50M -l 1400
```

ADVANCED

```
$ iperf3 -c fe80::1%eth0 -6
  # IPv6

$ iperf3 -c 192.168.1.20 --tos 184
  # DSCP EF class

$ iperf3 -c 192.168.1.20 -C cubic
  # congestion ctrl

$ iperf3 -c 192.168.1.20 -C bbr

$ iperf3 -c 192.168.1.20 -l 128
  # small packets
```

OUTPUT & AUTOMATION

```
$ iperf3 -c 192.168.1.20 -J
  # JSON output
$ iperf3 -c 192.168.1.20 --logfile a.log

$ iperf3 -c 192.168.1.20 -f m
  # format Mbit/s

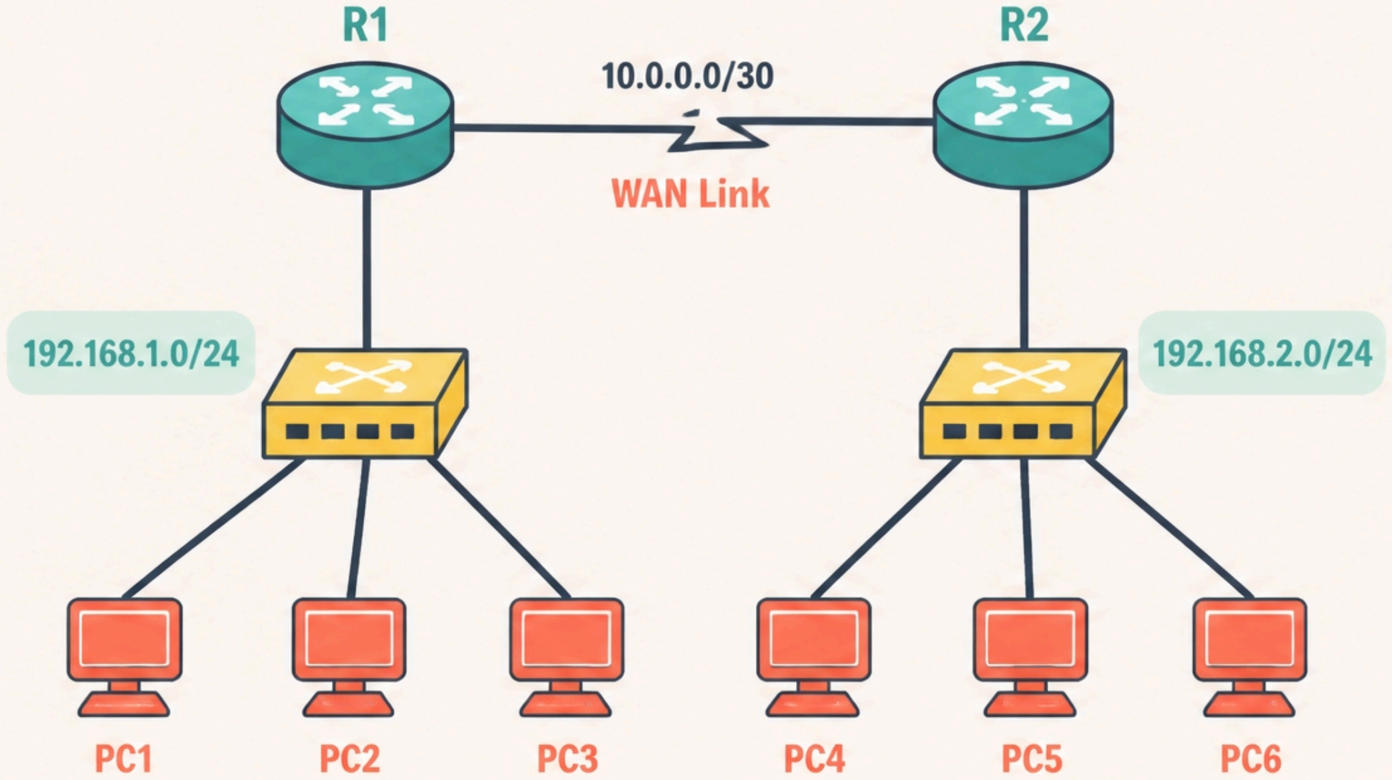
$ iperf3 -c 192.168.1.20 -V
  # verbose

$ iperf3 -c 192.168.1.20 --get-server-output
```

Pro tip: parallel streams (-P 4) can expose per-flow limits (e.g., single-CPU bottlenecks).

Section 4 • CISCO PACKET TRACER

Introductory Topology & Configuration

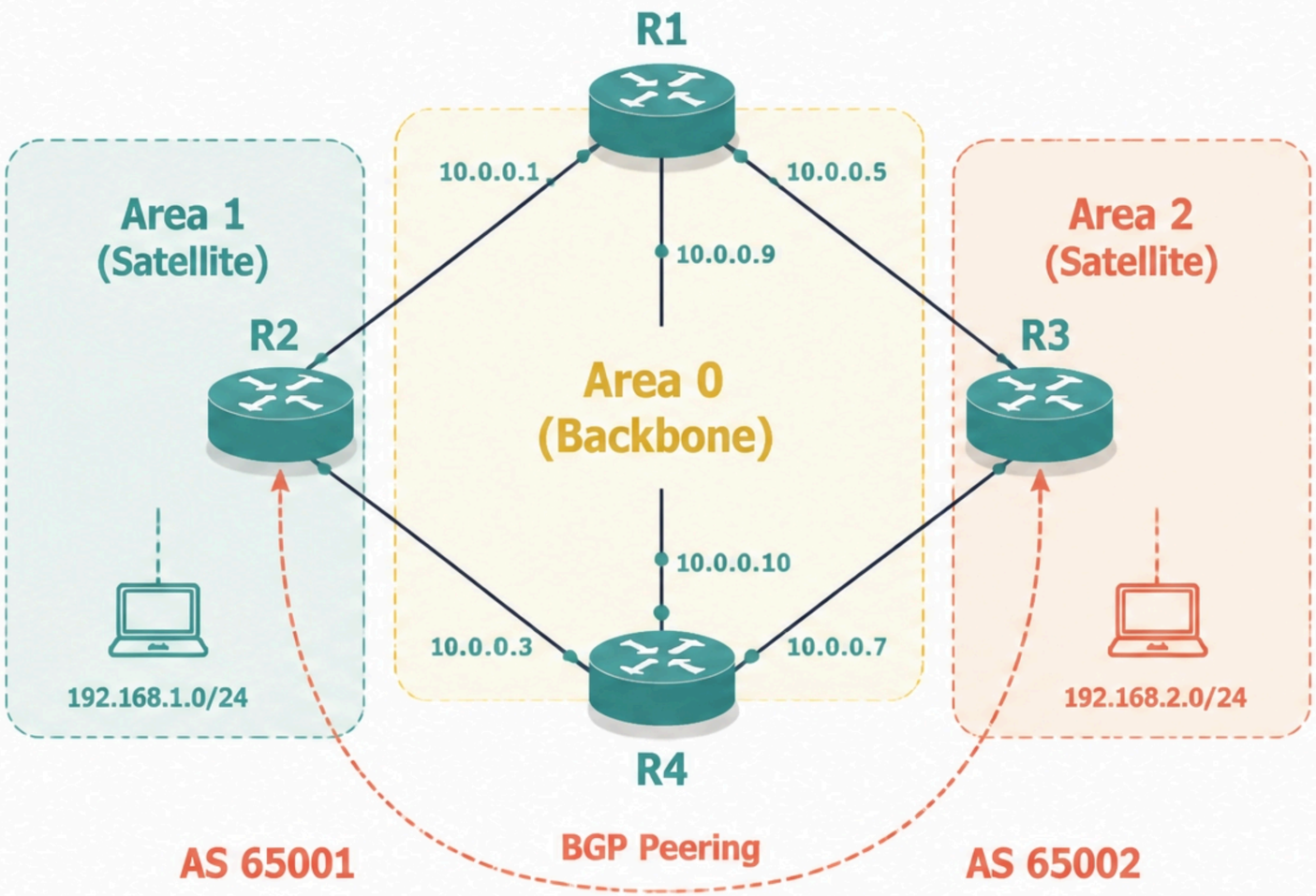


1. **SWITCHING & VLANs –** trunk, access, VTP
2. **ROUTING –** static, RIP, OSPF, EIGRP
3. **DHCP SERVER CONFIG –** pools, exclusions, reservations
4. **NAT & ACLs –** internet access & security filters

Great for CCNA prep • runs on Windows / macOS / Linux • free from Cisco Netacad

Section 5 · GNS3 · Advanced Routing Experiments

Real Cisco IOS · Multi-vendor topologies



BGP Peering Example

```
R1(config)# router bgp 65001
R1(config-router)# neighbor 10.0.0.2 remote-as 65002
R1(config-router)# network 192.168.1.0
```

Verify

```
R1#  show ip route
R1#  show ip ospf neighbor
R1#  show ip bgp summary
R1#  ping 192.168.2.1
R1#  traceroute 192.168.2.1
```

Multi-area OSPF · BGP · MPLS · frame relay — all real IOS

Section 8 · End-to-End Lab Recipes

Six hands-on labs to tie everything together

Learn. Build. Verify.

A LAB A · Packet Flow Correlation

Build a LAN in Packet Tracer · watch ARP + ICMP in Simulation · repeat on real host with Wireshark



Wireshark

```
1 ARP  Who has 192.168..2?
2 ARP  192.168!.2 is at ...
3 ICMP Echo (ping) request
4 ICMP Echo (ping) reply
...
```

B LAB B · Throughput Explained

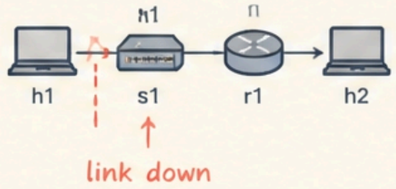
Run **iperf3 -c -P 4 -t 30** while tcpdump captures · inspect **tcp.analysis.retransmission** in Wireshark




```
# iperf3 client
iperf3 -c 10.0.0.2 -P 4 -t 30
# tcpdump capture
sudo tcpdump -i eth0 -w cap.pcap
```

C LAB C · Fault Injection

sudo mn --link tc,bw=10,delay=10ms,loss=1 · then link **s1 h1 down/up** · observe recovery




```
$ sudo mn --link
tc,bw=10,delay=10ms,loss=1

$ link s1 h1 down
$ link s1 h1 up
```

observe recovery

D LAB D · Static Routing (GNS3)

Two LANs, two routers · verify with **show ip route**, **ping**, **traceroute**




```
R1# show ip route
R1# ping 10.0.2.10
R1# traceroute 10.0.2.10
```

E LAB E · DNS Latency Study

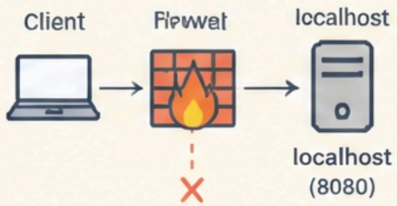
```
for i in $(seq 1 20); do
dig +noall +stats @8.8.8.8 example.com |
grep Query time; done
```




```
;; Query time: 12 msec
;; Query time: 13 msec
;; Query time: 11 msec
;; Query time: 15 msec
...
```

F LAB F · Firewall Verification

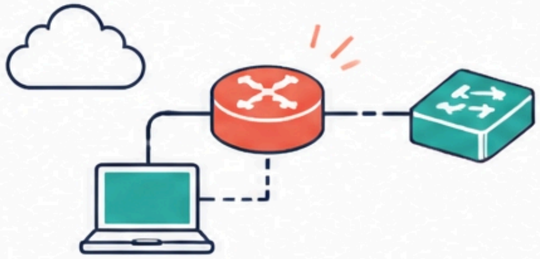
sudo iptables -A INPUT -p tcp --dport 8080 -j DROP · **nmap -p 8080 localhost** · compare open vs filtered




```
$ sudo iptables -A INPUT \
  -p tcp --dport 8080 -j DROP

$ nmap -p 8080 localhost

PORT     STATE
8080/tcp filtered
```



Section 9 . Compact Cheat Sheet

Everything on one page — print & pin above your desk



1. CAPTURE & ANALYSIS

```
wireshark &
tshark -i eth0 -Y "http"
sudo tcpdump -ni any port 53
capinfos capture.pcap
```



2. PERFORMANCE

```
iperf3 -s
iperf3 -c <server> -P 4 -t 30
iperf3 -c <server> -R
iperf3 -c <server> -u -b 50M
```



3. LINUX TOOLBOX

```
ip addr; ip route; ip neigh
ss -ltnp
ping -c 4 <host>
mtr <host>
dig <host>
sudo tcpdump -ni any
```



4. SIMULATION & EMULATION

```
# Packet Tracer: topology → IPs → Simulation
sudo mn --topo tree,depth=2,fanout=2 --test pingall

# GNS3: topology → CLI configure →
  capture on link
```



5. WINDOWS TOOLBOX

```
ipconfig /all
tracert 8.8.8.8
pathping google.com
Test-NetConnection example.com -Port 443
Get-NetTCPConnection -State Listen
```



6. DNS

```
dig +short example.com
dig -x 8.8..8.8
host -t MX example.com
dig +trace example.com
```







References & Acknowledgements

All content credited to original authors and open documenation

OFFICIAL DOCUMENTATION

1. **Wireshark User Guide & Display Filter Reference**
   wireshark.org
2. **Mininet Documentation**
   mininet.org
3. **iperf3 Manual**
   iperf.fr
4. **Cisco Packet Tracer**
   netacad.com
5. **GNS3 Documentation**
   docs.gns3.com
6. **Linux man pages**
   ip(8), ss(8), tcpdump(1), nmap(1), dig(1)
7. **Npcap Reference**
   npcap.com

BOOKS & COURSES

1. **Wireshark Network Analysis**
   Laura Chappell
2. **Computer Networking: A Top-Down Approach**
   Kurose & Ross
3. **TCP/IP Illustrated Vol.1**
   W. Richard Stevens
4. **Nmap Network Scanning**
   Gordon Lyon
5. **The Practice of System & Network Administration**
   Limoncelli et al.

> *This handbook synthesizes public documentation, open-source manuals, and classroom teaching notes. All trademarks belong to their respective owners.*

KEEP CAPTURING. KEEP LEARNING.

The wire never lies — and now you know how to listen.

